Harness Engineering in 2026: The Convergence of Physical Infrastructure and Autonomous Agent Scaffolding

A Comprehensive Technical Report

Google DeepMind Analysis Group
Compiled for PaperX

April 2026

Abstract

As of 2026, "Harness Engineering" has evolved into a critical dual-domain discipline. In software, it defines the structural environments, evaluation loops, and guardrails for autonomous AI coding agents at production scale. In aerospace and automotive, it represents the physical nervous system managing high-voltage power and high-speed data across zonal architectures. This paper provides an in-depth technical analysis of OpenAI's zero-code deployment experiment (1M lines via Codex, 3–7 engineers), and a comprehensive architectural dissection of Anthropic's Claude Code agent harness—now based on a complete 512K-line source leak including **40+ tool implementations**, a **4-tier hierarchical memory system**, a **full coordinator-mode multi-agent swarm protocol**, **lifecycle hooks framework**, **autonomous cron scheduling**, **git worktree isolation**, and **voice interface**. We argue both domains converge toward a unified philosophy: designing environments that prioritize *systemic reliability* over individual component performance.

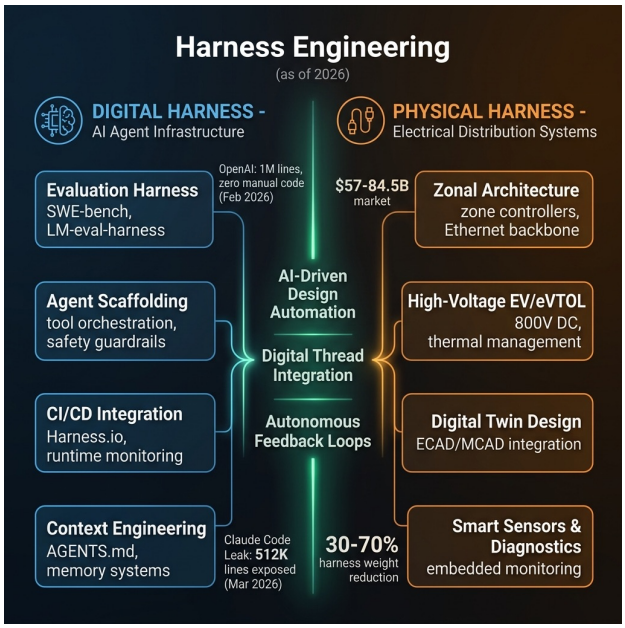

Figure 1: The convergence of Harness Engineering across physical systems and digital AI agent infrastructures in 2026.

1 Introduction

Historically, a "harness" referred to the physical wiring routing power and signals through complex machines. In 2026, the term has been co-opted by the AI community: to *harness* an LLM is to construct surrounding scaffolding—constraints, telemetry, evaluation frameworks, and safety guardrails—channeling raw token generation into reliable autonomous execution.

This convergence is not coincidental. Both disciplines face exponential combinatorial complexity managed through architectural constraint. A

modern vehicle contains over 5,000 wire segments under vibration and thermal stress. An AI agent harness orchestrates dozens of tool calls, filesystem mutations, and sub-agent delegations across multi-hour sessions. The failure modes are structurally identical—cascading faults from unconstrained interaction spaces.

This report surveys recent advancements in both domains, with particular depth on the Claude Code source analysis made possible by a second, more comprehensive data release in April 2026 exposing approximately **298 utility modules, 40+ tool implementations, and 20+ service subsystems**.

2 The Digital Harness: Agentic Scaffolding

2.1 OpenAI's Agent-First Experiment

In February 2026, OpenAI published a landmark retrospective [1] detailing a five-month experiment building a production platform using an "agent-first" methodology:

- **∼1 million lines of code** produced in 5 months.
- **1,500+ Pull Requests** opened and merged.
- Team grew from **3 to 7 engineers**.
- Throughput: **3.5 PRs per engineer per day**, with throughput *increasing* as the team grew.
- Codex tasks ran **6+ hours continuously**, often overnight.

The human role was redefined to "harness engineering"—setting up environments, defining constraints, and optimizing for agent readability. Architecture was strictly layered:

$$\text{Types} \rightarrow \text{Config} \rightarrow \text{Repo} \rightarrow \text{Service} \rightarrow \text{Runtime} \rightarrow \tag{1}$$

Key innovations included: the **Ralph Wiggum Loop** (self-review cycle), **Doc-Gardening Agent** (documentation drift correction), and **Entropy Management agents** enforcing **Golden Principles** against "AI residue." `AGENTS.md` served as a navigational map rather than encyclopedia.

Figure 2: The layered architecture of Claude Code, from input surfaces through orchestration, tool pool, and infrastructure.

2.2 Claude Code: Complete Architectural Dissection

The March 2026 source leak exposed ∼512K lines of TypeScript. A subsequent April 2026 data release provides the **complete implementation**, enabling definitive architectural analysis. The system comprises five layers (Figure 2) with a total of 35 subdirectories and 298+ utility modules.

2.2.1 QueryEngine: The Orchestration Core

At the heart of Claude Code sits the `QueryEngine` class (1,296 lines), owning the complete query lifecycle:

Listing 1: QueryEngine architecture (simplified)

```
class QueryEngine {
  mutableMessages: Message[]
  abortController: AbortController
  permissionDenials: SDKPermissionDenial[]
  totalUsage: NonNullableUsage
  readFileState: FileStateCache
  discoveredSkillNames: Set<string>

  async *submitMessage(prompt):
    AsyncGenerator<SDKMessage> {
    // Process user input
    // Fetch system prompt parts
    // Load skills + plugins (cache-only)
    // Enter query loop via query()
    // Yield SDK messages as async generator
  }
}
```

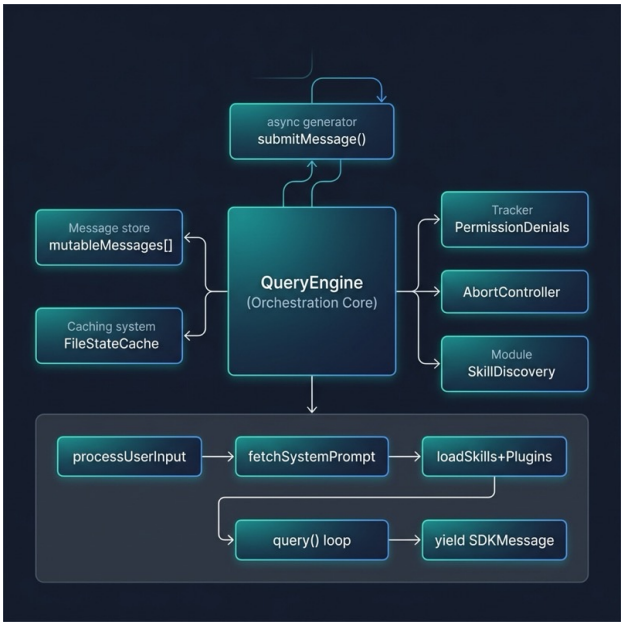

Figure 3: The QueryEngine orchestration core: async generator lifecycle with message store, file cache, permission tracking, and skill discovery.

Each `submitMessage()` call is a new turn within an ongoing conversation. The engine manages: mutable message arrays, file state caching, permission denial tracking, usage accumulation, skill discovery, structured output enforcement, and snip-boundary handling for bounded memory in headless sessions.

2.2.2 Compile-time Feature Flag System

Anthropic uses `bun:bundle` feature flags for dead code elimination:

$$\text{Build}(f) = \begin{cases} \text{Include}(M_f) & \text{if } f \in F_{\text{enabled}} \\ \varnothing & \text{otherwise} \end{cases} \quad (2)$$

2.2.3 Coordinator Mode: Full Orchestration Protocol

The most architecturally significant finding is the 370-line `coordinatorMode.ts` module, which exposes the **complete orchestration system prompt**. This is not a simple delegation mechanism—it is a formally specified multi-agent operating protocol:

Role Definition: The coordinator is explicitly defined as a *synthesizer*, not a delegator. The system prompt enforces: "*Never write 'based on your findings' or 'based on the research.' These phrases delegate understanding to the worker instead of doing it yourself.*"

Flag	Capability
`COORDINATOR_MODE`	Multi-agent swarm orchestration
`KAIROS`	Persistent daily-log assistant
`PROACTIVE`	Autonomous wake-up execution
`HISTORY_SNIP`	Snip-based context compaction
`EXTRACT_MEMORIES`	Background memory extraction
`TEAMMEM`	Shared team memory system
`COMMIT_ATTRIBUTION`	Git commit author tracking
`BRIDGE_MODE`	IDE integration (VS Code, JetBrains)
`CHICAGO_MCP`	Computer-use / screen control
`VOICE_MODE`	Native audio I/O pipeline
`BUDDY`	Companion sprite UI

Table 1: Feature flags discovered in the Claude Code leak.

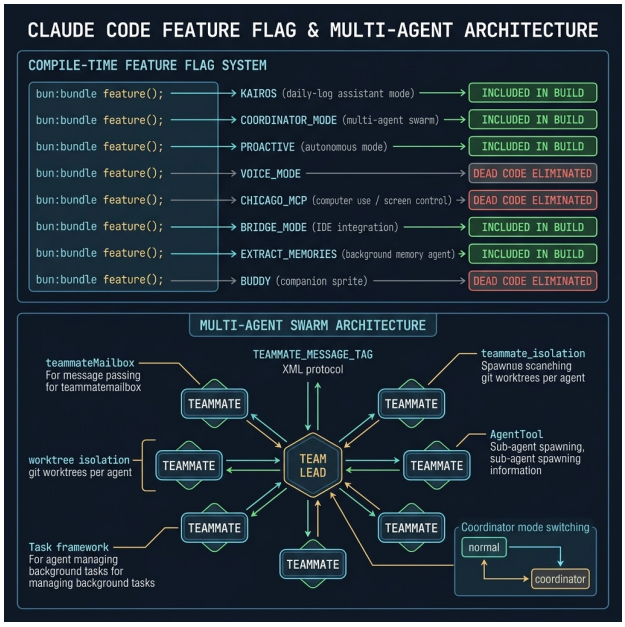

Figure 4: Compile-time feature flag system and multi-agent topology.

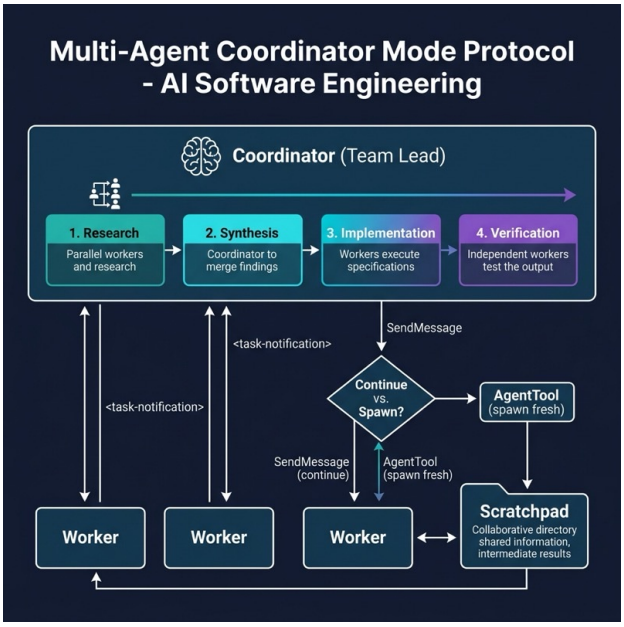

Figure 5: The Coordinator Mode multi-agent protocol: 4-phase workflow with continue-vs-spawn decision logic, XML task notifications, and shared scratchpad.

Phase-Based Workflow: Tasks follow four phases:

1. **Research**: Workers investigate in parallel. Read-only tasks run concurrently without constraints.
2. **Synthesis**: The coordinator reads findings, identifies approach, crafts implementation specs with *specific file paths, line numbers, and exact changes*.
3. **Implementation**: Workers execute per-spec. Write-heavy tasks serialized per-file-set.
4. **Verification**: Independent workers prove code works, not just confirm existence.

Worker Communication: Workers receive tasks via `AgentTool` spawning, with results arriving as XML `<task-notification>` messages:

Listing 2: Task notification protocol

```
<task-notification>
  <task-id>{agentId}</task-id>
  <status>completed|failed|killed</status>
  <summary>{outcome description}</summary>
  <result>{agent's final response}</result>
  <usage>
    <total_tokens>N</total_tokens>
    <tool_uses>N</tool_uses>
    <duration_ms>N</duration_ms>
  </usage>
</task-notification>
```

Situation	Action	Rationale
Research explored exactly the files needing editing	Continue	Context reuse
Broad research, narrow implementation	Spawn fresh	Clean context
Correcting a failure	Continue	Error context
Verifying another worker's code	Spawn fresh	Fresh eyes
Wrong approach entirely	Spawn fresh	Avoid anchoring

Table 2: Coordinator-mode worker continuation decision matrix.

Continue vs. Spawn Decision: The protocol includes a formal decision matrix (Table 2).

Shared Scratchpad: When the `tengu_scratch` feature gate is enabled, a shared scratchpad directory provides durable cross-worker knowledge persistence without permission prompts.

2.2.4 4-Tier Hierarchical Memory System

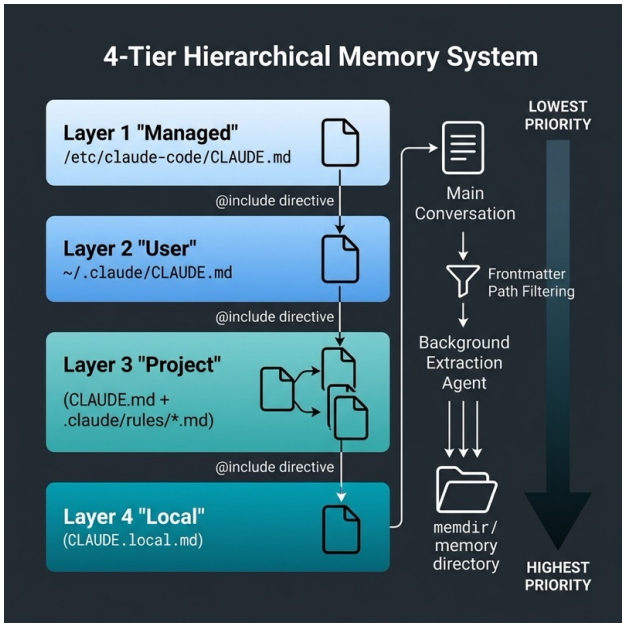

Figure 6: The 4-tier hierarchical memory system with @include directives, frontmatter filtering, and background extraction agent writing to memdir/.

The 1,480-line `claudemd.ts` module reveals a sophisticated 4-tier memory hierarchy (Table 3), with files loaded in reverse priority order (latest = highest priority):

Key mechanisms include:

Tier	Path	Scope
Managed	`/etc/claude-code/CLAUDE.md`	All users
User	`~/.claude/CLAUDE.md`	Private global
Project	`CLAUDE.md`, `.claude/CLAUDE.md`, `.claude/rules/*.md`	Repo-specific
Local	`CLAUDE.local.md`	Private project

Table 3: Claude Code 4-tier memory hierarchy.

- **@include directives**: Files can reference others via `@./path`, `@~/path`, or `@/absolute/path`, with circular reference prevention and max depth of 5.
- **Frontmatter path filtering**: Files use YAML frontmatter with `paths` to scope rules to specific file globs.
- **HTML comment stripping**: Block-level comments are stripped via marked lexer, preserving inline code.
- **Instruction priority**: "*These instructions OVERRIDE any default behavior and you MUST follow them exactly as written.*"

2.2.5 Background Memory Extraction Agent

The 616-line `extractMemories.ts` implements an autonomous background agent using the **forked agent pattern**—a process fork sharing the parent's prompt cache:

- Runs once at end of each complete query loop (no pending tool calls).
- Uses cursor-based incremental extraction: only processes messages since last extraction.
- Mutual exclusion with main agent's memory writes—skips if main agent already wrote to memory files.
- Hard cap of **5 turns** to prevent rabbit-holes.
- Sandboxed: only `Read/Grep/Glob`, read-only `Bash`, and `Edit/Write` within the memory directory.
- Supports both private (`autoMem`) and shared team (`teamMem`) memory paths.

2.2.6 Lifecycle Hooks Framework

The 5,023-line `hooks.ts` module implements a comprehensive lifecycle hooks system—user-defined shell commands executed at prescribed points:

Hooks output structured JSON with `decision` (approve/block), `permissionDecision` (allow/deny/ask), `continue` flags,

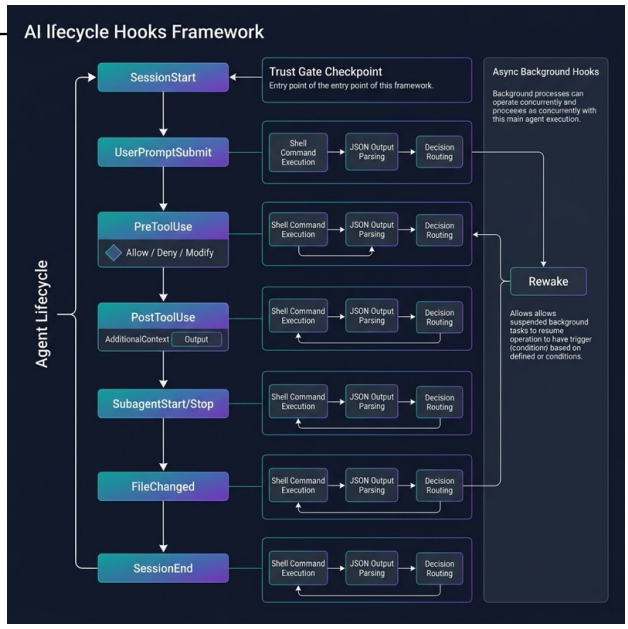

Figure 7: The lifecycle hooks framework: trust-gated shell command execution with JSON output parsing, decision routing, and async rewake mechanism.

Hook Event	Trigger Point
`PreToolUse`	Before tool execution (can block/allow/modify input)
`PostToolUse`	After tool completion
`UserPromptSubmit`	On user message submission
`SessionStart/End`	Session lifecycle boundaries
`SubagentStart/Stop`	Sub-agent lifecycle
`PermissionRequest`	Permission decision interception
`Elicitation`	MCP elicitation handling
`InstructionsLoaded`	CLAUDE.md files loaded
`FileChanged`	File system mutation detected
`ConfigChange`	Configuration update

Table 4: Claude Code lifecycle hook events.

`updatedInput` for tool input modification, and `hookSpecificOutput` for event-type-specific responses. All hooks are trust-gated: they require workspace trust dialog acceptance before execution.

2.2.7 Autonomous Cron Scheduling

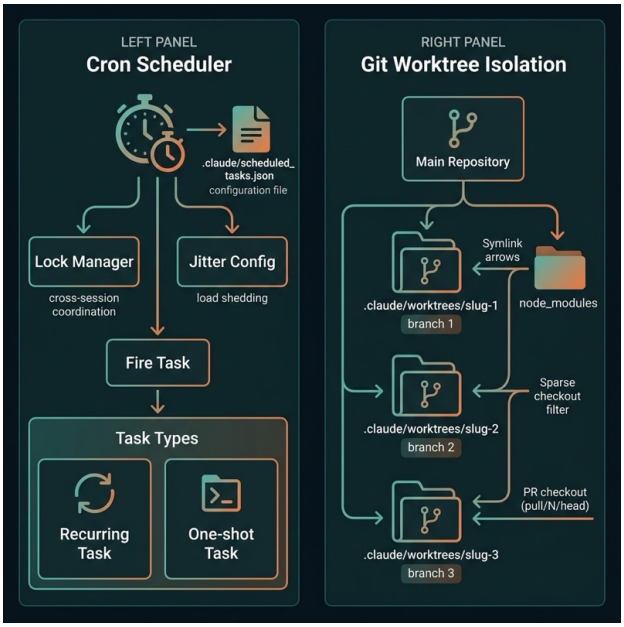

Figure 8: Autonomous operations: cron scheduling with lock-based coordination (left) and git worktree isolation with sparse checkout and symlinking (right).

The 566-line `cronScheduler.ts` reveals that Claude Code implements autonomous task scheduling:

- Tasks stored in `.claude/scheduled_tasks.json` with cron expressions.
- Lock-based coordination across concurrent sessions prevents double-firing.
- Supports both one-shot and recurring tasks with jittered next-fire times.
- Missed task detection with user confirmation before execution.
- Recurring tasks auto-expire after configurable `recurringMaxAgeMs`.
- 1-second polling loop with `chokidar` file watching.

This represents a fundamental capability shift: the agent can schedule future autonomous actions, moving from reactive to proactive operation.

2.2.8 Git Worktree Isolation

The 1,520-line `worktree.ts` implements complete filesystem isolation for parallel development:

- Named worktrees under `.claude/worktrees/<slug>`.
- Sparse checkout support via `settings.worktree.sparsePaths`.
- Directory symlinking (`node_modules`) to prevent disk bloat.
- `.worktreeinclude` file for copying gitignored assets.
- Hook-based VCS extensibility for non-git systems.
- PR checkout integration: `git fetch origin pull/N/head`.
- Tmux session creation for terminal multiplexing.
- Settings and git hooks propagation from main repository.

2.2.9 Voice Interface and Multimodal Input

The voice service implements push-to-talk audio input with a three-tier fallback chain:

1. **Native cpal**: In-process audio capture via CoreAudio (macOS), ALSA (Linux), or Windows audio.
2. **arecord**: ALSA-based fallback for Linux/WSL2+WSLg.
3. **SoX rec**: Universal fallback with silence detection.

Recording captures 16kHz, 16-bit signed, mono PCM. The `voiceStreamSTT.ts` (21KB) handles speech-to-text streaming, while `voiceKeyterms.ts` manages voice-specific keyword handling.

2.2.10 Tool Catalog: 40+ Implementations

The complete tool directory reveals 40 distinct tool implementations, categorized as:

The `ToolSearchTool` implements dynamic tool loading via `tool_reference` content blocks, deferring MCP tool definitions until needed. Auto-threshold triggers deferral when tool descriptions exceed 10% of context window tokens.

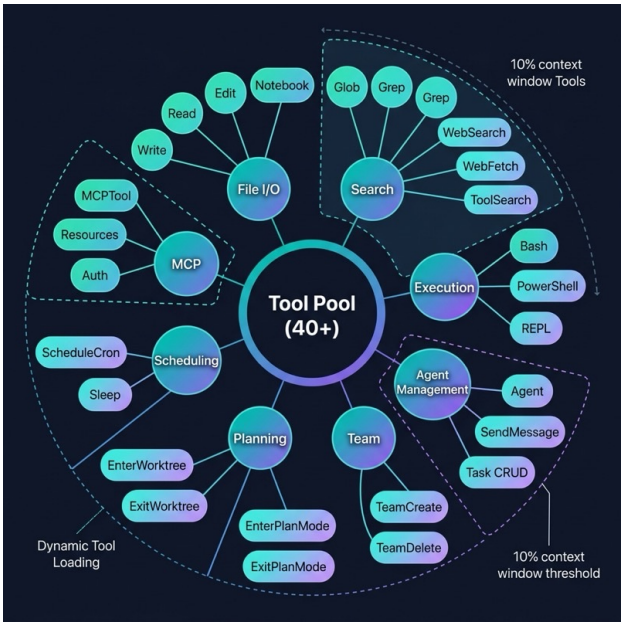

Figure 9: The complete 40+ tool catalog organized by category, with dynamic deferred loading via tool_reference blocks triggered at 10% context window threshold.

Category	Tools
File I/O	FileRead, FileWrite, FileEdit, NotebookEdit
Search	Glob, Grep, WebSearch, WebFetch, ToolSearch
Execution	Bash, PowerShell, REPL
Agent Mgmt	Agent, SendMessage, TaskCreate/Get/List/Output/Stop
Team	TeamCreate, TeamDelete
Planning	EnterPlanMode, ExitPlanMode
Isolation	EnterWorktree, ExitWorktree
MCP	MCPTool, ListMcpResources, ReadMcpResource, McpAuth
Scheduling	ScheduleCron, Sleep
Other	Config, Brief, Skill, AskUserQuestion, TodoWrite, SyntheticOutput, RemoteTrigger

Table 5: Complete Claude Code tool catalog (40+ tools).

2.2.11 Bridge and Remote Execution

The bridge module enables remote operation with:

- Up to **32 concurrent sessions** per environment.
- JWT token refresh 5 minutes before expiry.
- Heartbeat protocol preventing PEL eviction.
- Each session in isolated git worktree.

2.3 Evaluation Frameworks

3 The Physical Harness

3.1 Zonal Architecture and Ethernet Backbones

The industry is transitioning from domain-based to **Zonal Architectures**, achieving **30% to 70% reduction** in harness weight. The critical enabler is Automotive Ethernet:

- **TSN**: Deterministic bounded-latency for ADAS sensor fusion.
- **10BASE-T1S**: Single-pair multi-drop replacing CAN/LIN.
- **Multi-Gig Backhaul**: 2.5–10 Gbps for aggregated sensor streams.

3.2 High-Voltage eVTOL Challenges

eVTOL 800V+ DC distribution at altitude faces corona discharge:

$$V_{\text{corona}} \propto \delta \cdot r \cdot \ln\left(\frac{D}{r}\right) \cdot E_0 \tag{3}$$

where δ is relative air density, dropping at cruise altitude (2,000–5,000 ft) to reduce V_{corona} below operating voltage. Engineers balance safety margins with SWaP constraints via corona-resistant PTFE insulators and hybrid Al-Cu conductors.

4 The Convergence

The digital and physical domains are operationally converging through AI-driven digital twins. ECAD/MCAD workflows simulate physical routing alongside software systems. Generative AI optimizes BOM extraction, route planning, and topology generation—using software harnesses to design physical ones.

This creates a feedback loop: physical constraints (bend radii, thermal derating) become AI

Framework	Evaluation Target	Methodology	Harness Role
SWE-bench Verified	Code generation agents	End-to-end PR resolution	Task scaffolding, repo setup
Agent Trajectory Eval	Multi-step reasoning	Semantic validity per tool call	Step-level guardrails
Guardrails-as-Code	Production runtime	SLM-powered interception	Real-time safety harness
LM-eval-harness	Foundation models	Standardized prompt suites	Tokenizer + sampling confi

Table 6: Evolution of evaluation frameworks in the agentic era.

optimization parameters, while software telemetry (tool-use latency, context utilization) drives evaluation environment design.

5 Comparative Analysis

Table 7 reveals Claude Code as architecturally the most complex agent harness publicly analyzed. Its 40+ tool implementations, 4-tier memory hierarchy, lifecycle hooks, cron scheduling, and worktree isolation represent an operating-system-level approach to agent infrastructure—far beyond conversational wrappers.

6 Conclusion

Whether routing 10 Gbps sensor data through an Ethernet backbone or channeling LLM reasoning through evaluation guardrails, harness engineering in 2026 is about *managing complexity through architectural constraint*.

The OpenAI experiment demonstrated sustainable 3.5 PRs/day/engineer with mature harness infrastructure. The Claude Code analysis—now based on the **complete source**—reveals an agent operating system: a QueryEngine orchestration core, coordinator-mode swarm protocol with formal synthesis requirements, 4-tier hierarchical memory with background extraction agents, 5,000-line lifecycle hooks framework, autonomous cron scheduling, git worktree filesystem isolation, voice input, and 40+ tool implementations with dynamic deferred loading. In the physical domain, zonal architectures and Ethernet backbones mirror the same layered, constraint-driven philosophy.

The future lies in continued hybridization, where autonomous software agents design physical electrical architectures—and where physical constraints feed back into evaluation harnesses training the next generation of AI engineers.

References

[1] OpenAI Engineering (Feb. 2026). *Harness Engineering: Building Software in an Agent-First World*. OpenAI Blog.

[2] Security Research Community (Mar.–Apr. 2026). *Anthropic Claude Code CLI Source Exposure via npm Source Maps*. Independent disclosure.

[3] Jimenez et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024.

[4] IEEE 802.1 Working Group (2025). *Time-Sensitive Networking for Automotive Ethernet*. IEEE Standards.

[5] SAE International (2025). *ARP6336: High-Voltage Wiring Systems for eVTOL Aircraft*. SAE Recommended Practice.

[6] Bosch Engineering (2025). *Zonal E/E Architectures for Software-Defined Vehicles*. White Paper.

[7] Gao et al. (2023). *A Framework for Few-Shot Language Model Evaluation*. Zenodo.

[8] Anthropic (2024). *Model Context Protocol Specification*. Open Standard.

[9] Confident AI (2025). *DeepEval: Agent Trajectory Evaluation Framework*. Documentation.

Dimension	**OpenAI Codex**	**Claude Code**	**Cursor**
Execution Model	Cloud sandboxed	Local-first CLI	IDE-embedded
Multi-agent	Agent-to-agent review	Coordinator swarm (workers)	Single-agent
Memory System	Repo-as-context	4-tier CLAUDE.md + memdir	Codebase indexing
Context Strategy	AGENTS.md map	@include hierarchy + rules/	Retrieval-augmented
Safety Model	CI linters + Golden Principles	Hooks + trust gates + sandbox	User approval
Scheduling	Task queues	Cron scheduler + PROACTIVE	N/A
Isolation	N/A	Git worktrees + sparse checkout	N/A
Tool Count	Undisclosed	40+ (deferred loading)	$\sim$10
Voice Input	No	Native cpal + STT streaming	No
Max Autonomy	6+ hours	Cron-scheduled + PROACTIVE	User-initiated

Table 7: Comprehensive comparison of agent harness platforms, Q2 2026.